

Static Analysis Android Application

Day2:Lab manual

Table of Contents

	page no
1. Objectives	3
2. Prerequisites	3
3. Setup Tools for Windows	3
3.1. Android Studio	3
3.2. AVD Emulator	3
3.3. Jadx	4
3.4. Apktool	6
3.5. Adb Tool	11
3.6. Python 3.12	15
3.7. MobSF	17
4. Setup Tools for Linux (Ubuntu)	18
4.1. Android Studio (Linux)	18
4.2. AVD Emulator (Linux)	18
4.3. Jadx (Linux)	18
4.4. Apktool	21
4.5. Adb Tool	23
4.6. Python 3.12	26
4.7. MobSF	28
5. Static Analysis for Windows, Linux	29
5.1 Select the Target APK	29
5.2 Open the APK in JADX	29
5.3 Search for Important Security Keywords	32
5.4 Decode the APK Using APKTool	35
5.5 Review AndroidManifest.xml	37
5.6 Review Deep Links and WebViews	39
5.7 Review Resources and Sensitive Files	40
5.8 Scan the APK Using MobSF	41
5.9 Identify Security Findings and Prepare Observations	42

1. Objectives

The objective of this lab is to install and configure different tools used in Static and Dynamic Analysis of the application for **Windows Operating System and Linux**.

2. Prerequisites

Operating System	Windows 10/11 (64-bit) or Ubuntu Linux 20.04/22.04/24.04
Hardware	● CPU: x86_64 CPU architecture ● RAM: 8 GB RAM or more ● Disk Space: 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
Tools Required	Android Studio, AVD Emulator, Jadx, Apktool, Mobsf

3. Setup Tools for Windows

3.1. Android Studio

The setup process is already completed in the previous lab.

3.2. AVD Emulator

The setup process is already completed in the previous lab.

3.3. Jadx

3.3.1. Download the VAPT_Android.zip file from the following link: [DownloadLink](#)

3.3.2. Locate the downloaded **VAPT_Android_Tools.zip** file, right-click on it, and select **Extract All** to extract its contents.

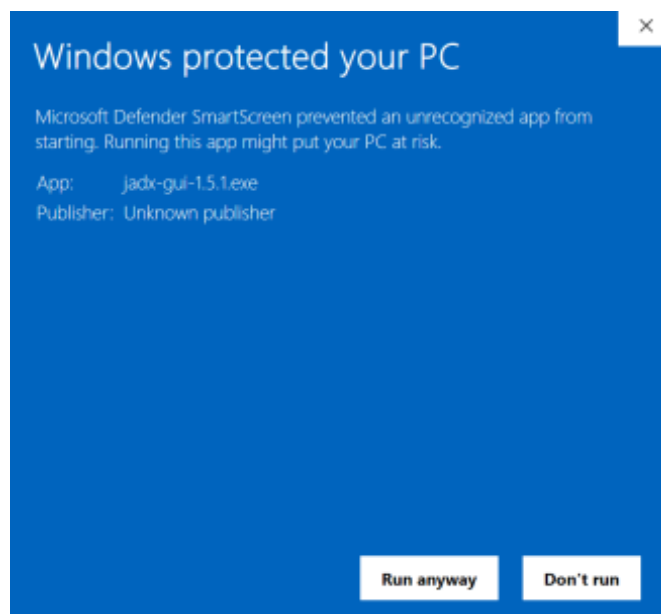
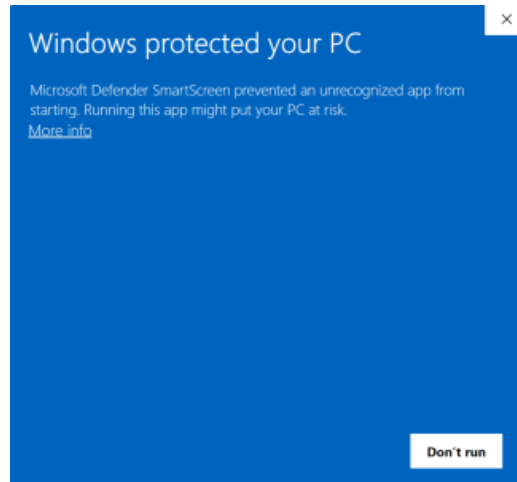
Name	Date modified	Type	Size
VAPT_tools_linux	16-09-2026 23:35	File folder	
VAPT_tools_windows	16-09-2026 23:54	File folder	

3.3.3. GO inside VAPT_Android/VAPT_Windows_Tools/jadx folder

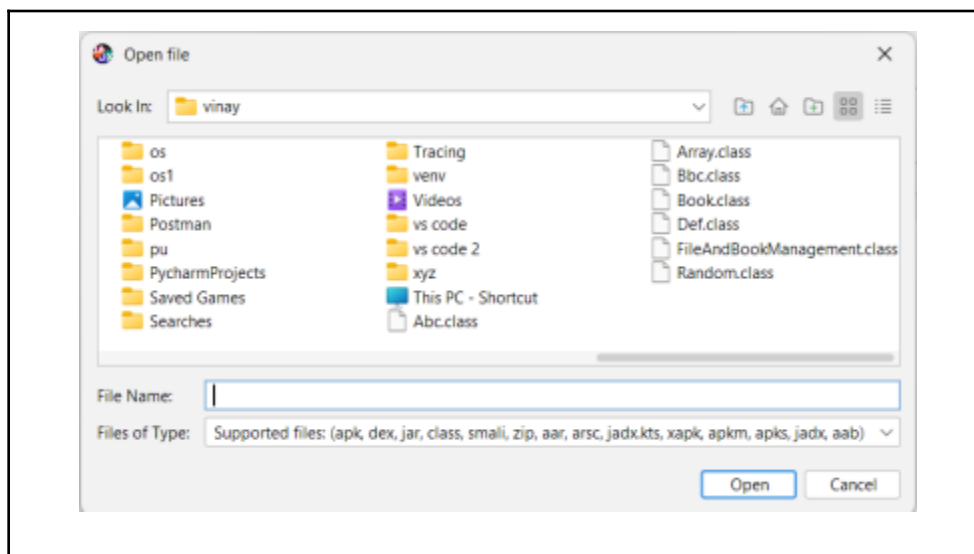
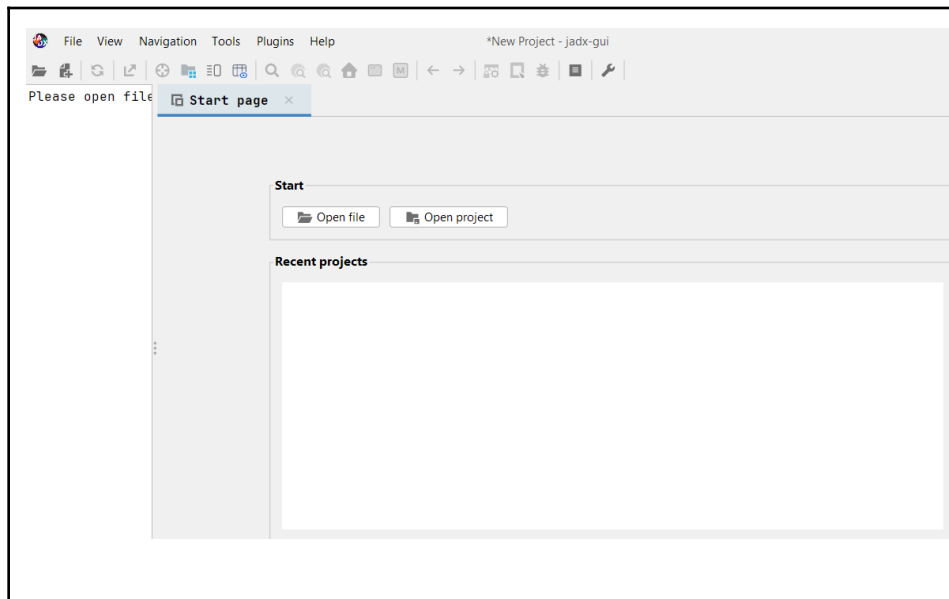
Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
jre	File folder					16-09-2026 09:43
lib	File folder					16-09-2026 09:43
jadx-gui-1.5.6	Application	38 KB	No	79 KB	53%	16-09-2026 09:43

3.3.4. Double click **jadx-gui-1.5.6** binary to open the jadx tool

If you got some prompt as shown below, click on **“More info”** then click **“Run anyway”**



3.3.5. Now click on “**Open file**” from the window and select any **.apk** file for decompilation

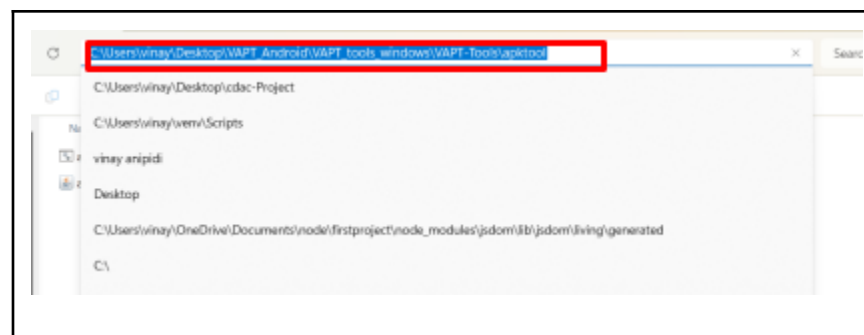


3.4. Apktool

3.4.1. GO inside **VAPT_Android_Tools/VAPT_Tools_windows/apktool/** folder

Name	Date modified	Type	Size
apktool	16-09-2026 23:54	Windows Batch File	3 KB
apktool_3.0.3	16-09-2026 23:54	Executable Jar File	15,116 KB

3.4.2. Copy the path of the folder and add the path to the Environment Variables



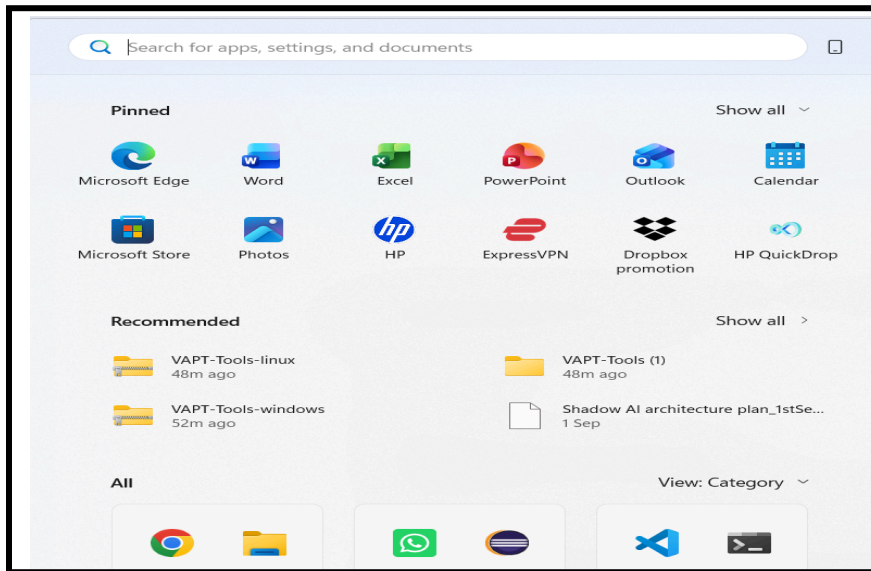
3.4.3. In this case the path is :

`C:\Users\DELL\Desktop\VAPT_Android_Tools\apktool`

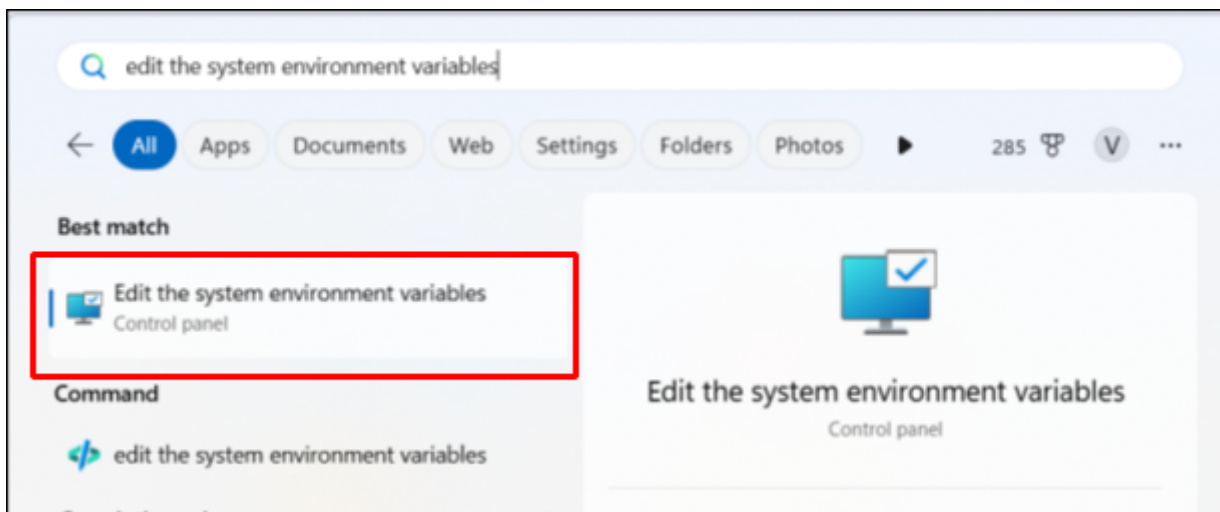
3.4.4. Open System Environment Variables — Windows

To add the **APKTool folder path** to Windows PATH, open the Environment Variables settings using the following steps:

1. Click the **Start** menu.

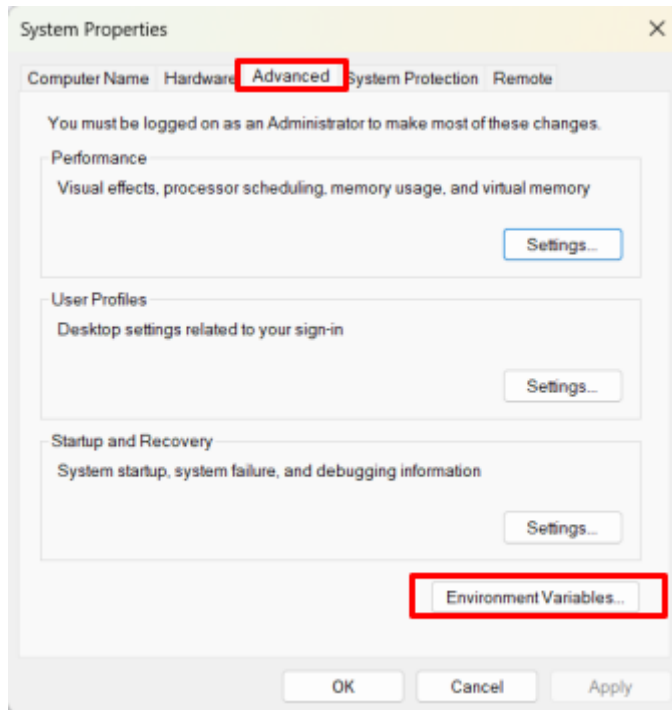


2. Type:
edit the system environment variables

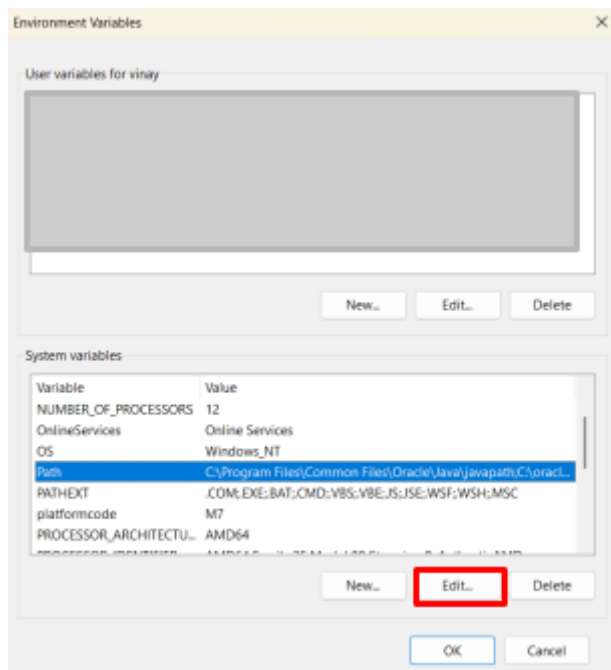


3. Select **Edit the system environment variables** from the search results.
4. The **System Properties** window opens on the **Advanced** tab.

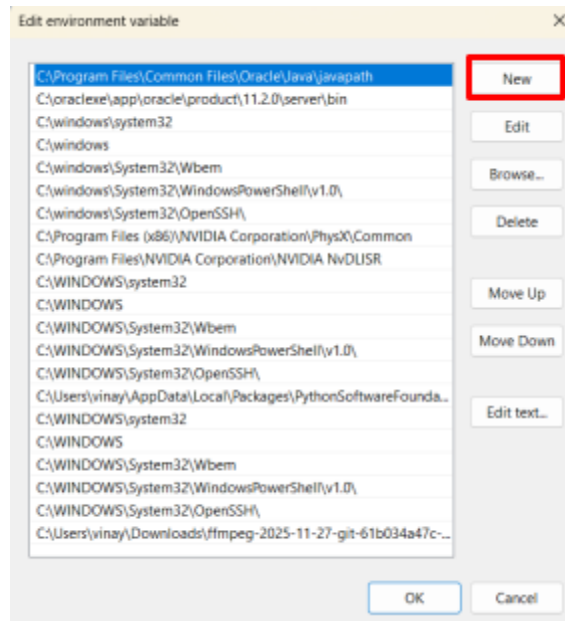
3.4.5. Click on Environment Variables



3.4.6. Select path and click edit



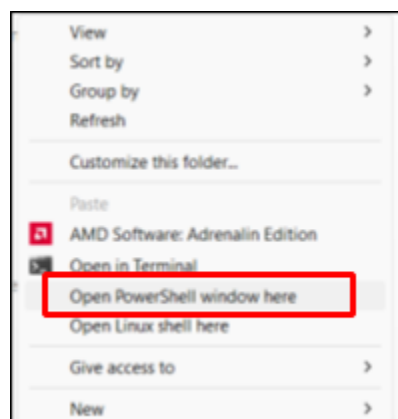
3.4.7. Add new and paste the previously copied path



3.4.8. Click OK and again OK on the second page

3.4.9. Now go to any folder do

Shift + right click, select **“open powershell windows here”**



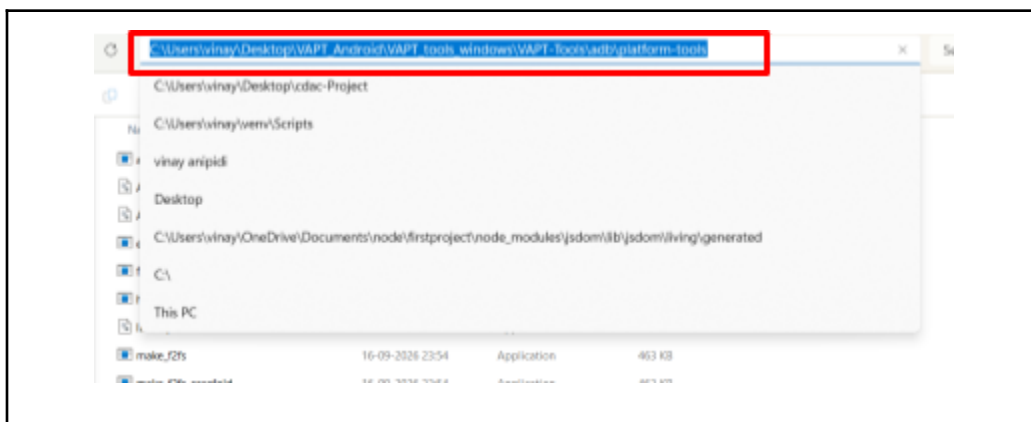
3.4.10. Run “apktool –version”

```
PS C:\Windows> apktool --version
3.0.3
Press any key to continue . . .
```

3.5. Adb Tool

3.5.1. GO inside VAPT_Android_Tools/platform-tools/ folder

3.5.2. Copy the folder path



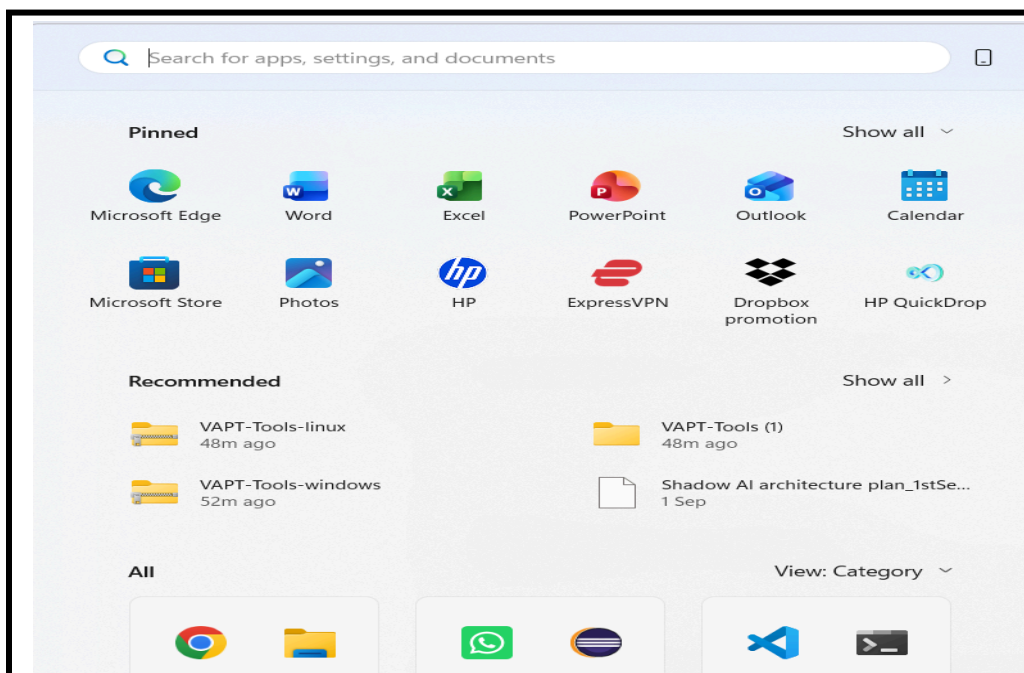
3.5.3. Here it is

C:\Users\vinay\Desktop\VAPT_Android_Tools\VAPT_tools_windows\VAPT-Tools\adb\platform-tools

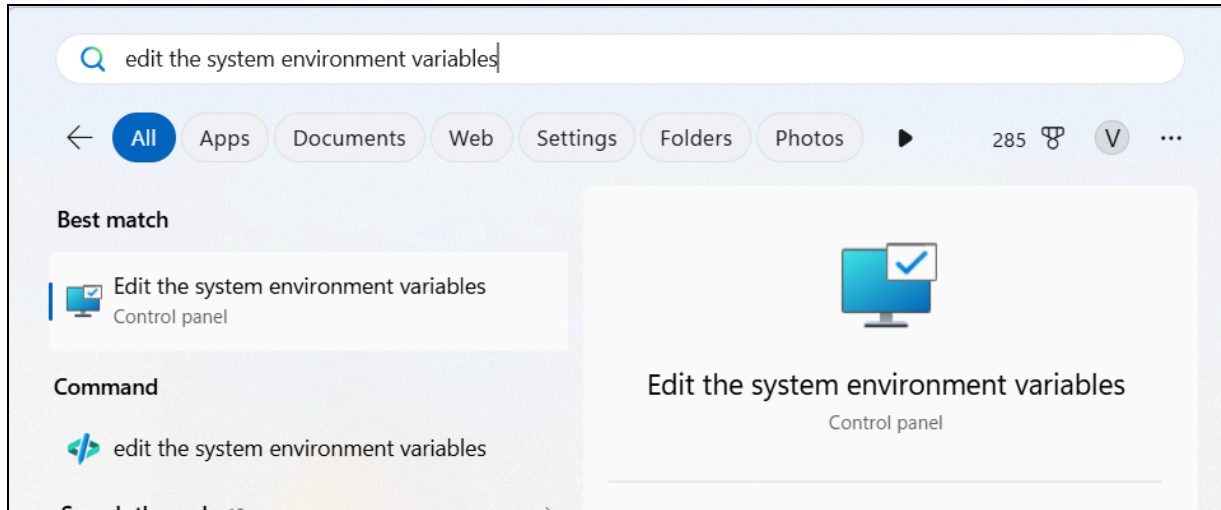
3.5.4. Open System Environment Variables — Windows

To add the **APKTool folder path** to Windows PATH, open the Environment Variables settings using the following steps:

1. Click the **Start** menu.

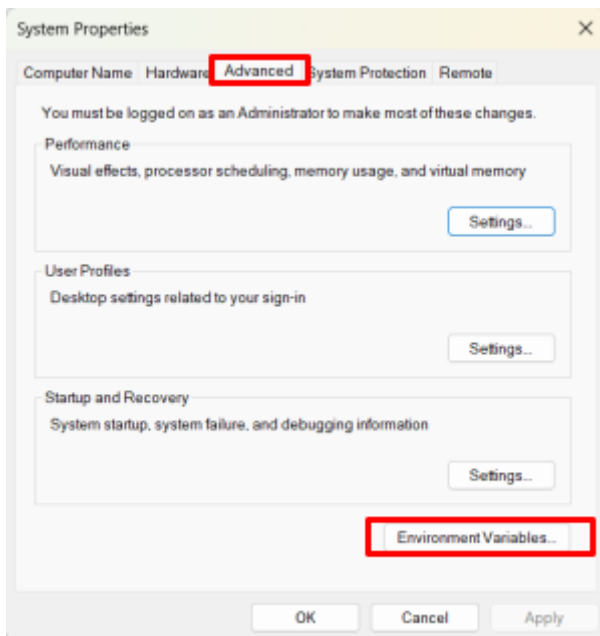


2. Type:
edit the system environment variables

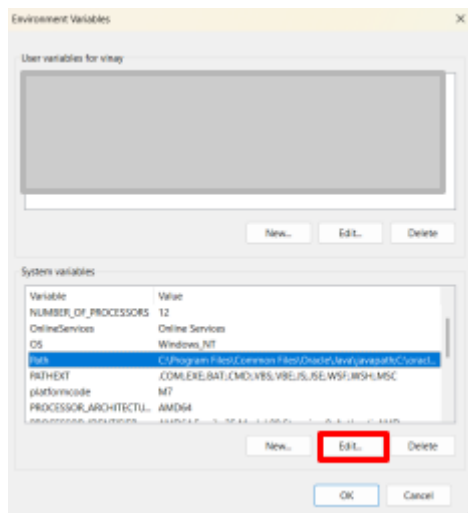


3. Select **Edit the system environment variables** from the search results.
4. The **System Properties** window opens on the **Advanced** tab.

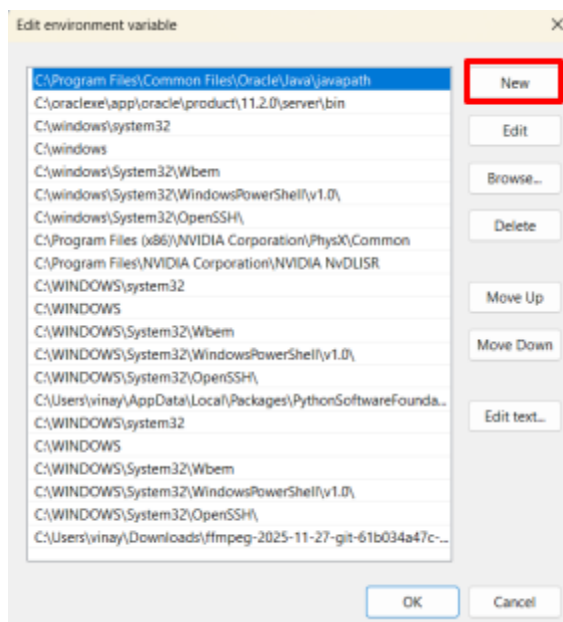
3.5.5. Click on **Environment Variables**



3.5.6. Select path and click edit



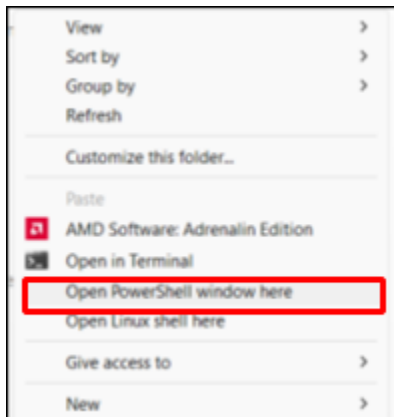
3.5.7. Add new and paste the previously copied path



3.5.5 Click **OK** and again **OK** on the second page

3.5.6. Now go to any folder do

Shift + right click, select “**open powershell windows here**”



3.5.7. Run “**adb —version**”

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\adb\platform-tools> adb --version
Android Debug Bridge version 1.0.41
Version 37.0.1-15733141
Installed as C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\adb\platform-tools\adb.exe
Running on Windows 10.0.26200
```

3.6. Python 3.12

3.6.1. Go to the official Python download page: [Python 3.12](https://www.python.org/downloads/windows/)

3.6.2. Scroll down to the “**Files**” section and download the appropriate installer based on your system(**win + i > system > about**)

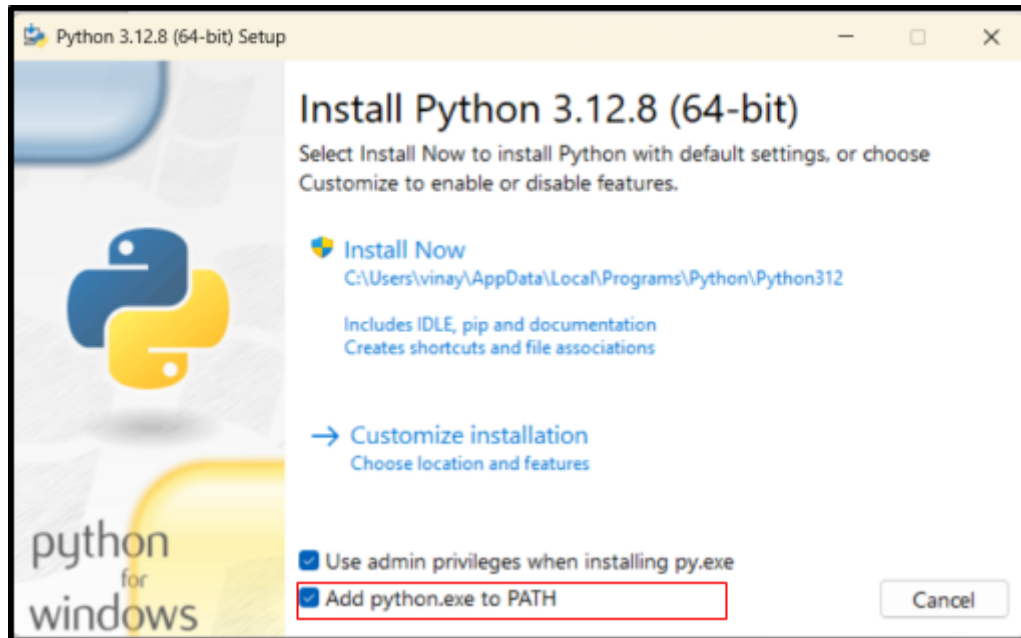
Note : intel or amd install 64 bit , ARM install ARM64

Version	Operating system	Description	File size	Sigstore	SBOM	GPG	MD5 checksum
Gzipped source tarball	Source release		25.8 MB	.sigstore	SPOX	SIG	304473cf367fa65e450edf4b86b55fcc
XZ compressed source tarball	Source release		19.5 MB	.sigstore	SPOX	SIG	d46e5bf9f2e596a3ba45fc0b3c053dd2
macOS 64-bit universal2 installer	macOS	for macOS 10.13 and later	44.0 MB	.sigstore		SIG	dc762fdc78e9cfecf516db31854de9fd
Windows installer (64-bit)	Windows	Recommended	25.8 MB	.sigstore	SPOX	SIG	2f2ab2472a6aa29f875c72c58f58f4b
Windows installer (32-bit)	Windows		24.6 MB	.sigstore	SPOX	SIG	745f11c8474893da55e5966173375cc8
Windows installer (ARM64)	Windows	Experimental	25.1 MB	.sigstore	SPOX	SIG	ff0d440c2cc4aadff81c9e247682bfa9
Windows embeddable package (64-bit)	Windows		10.6 MB	.sigstore	SPOX	SIG	1e86b04bc7d27c5c06edf8f617e1184e
Windows embeddable package (32-bit)	Windows		9.5 MB	.sigstore	SPOX	SIG	cd4a16b1d27540b84e7a44327f69ee5a
Windows embeddable package (ARM64)	Windows		9.9 MB	.sigstore	SPOX	SIG	a1631f5cb0b3d5d1a27b5c3edc0f00e3

3.6.3. Run the Installer

After the installer is **downloaded**, **double-click** it to start the installation Process.

Important: Make sure to check the box that says "Add Python 3.6 to This environment PATH" at the bottom of the installation window. This ensures that Python will be available globally via the command line.



Then click **Install Now**, and then click **Yes**, if any prompt comes, and finally close the windows.

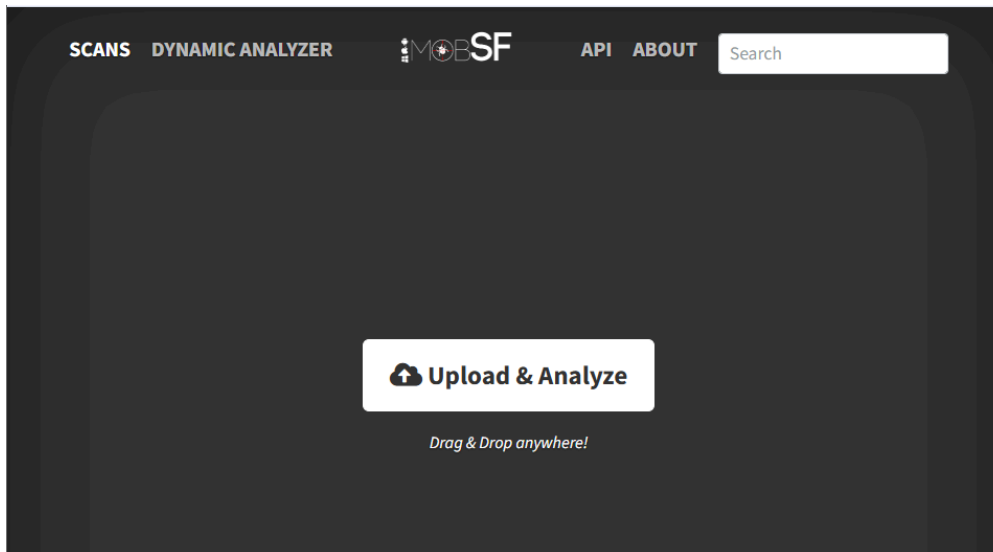
3.6.4. Check the installation by opening a **powershell window** and then writing the below command

```
PS C:\Users\vinay\Desktop\VAPT_Android\VAPT_tools_windows\VAPT-Tools\adb\platform-tools> python --version
Python 3.12.8
```

3.7. MobSF

3.8.1. Go to <https://mobsf.live/>

3.8.2. Upload **.apk file** directly from the web application



4. Setup Tools for Linux (Ubuntu)

4.1. Android Studio (Linux)

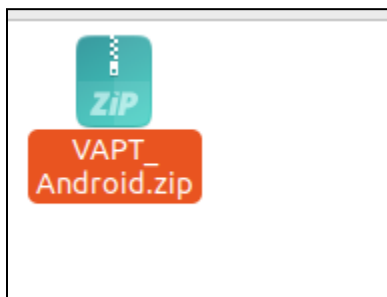
The setup process is already completed in the **previous lab**.

4.2. AVD Emulator (Linux)

The setup process is already completed in the **previous lab**.

4.3. Jadx (Linux)

4.3.1. Downloaded the **VAPT_Android_Tools.zip** file from the following link: [DownloadLink](#)



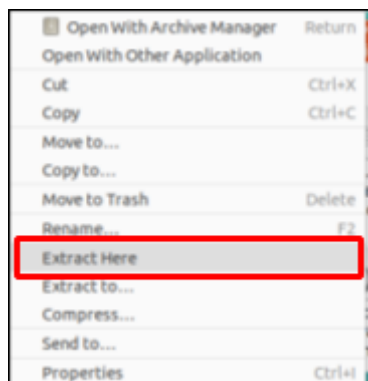
4.3.2. Open the **Downloads** folder.

1.Right-click the downloaded file:

VAPT_Tools_Linux.zip

2.Select:

Extract Here



Or open a terminal in the Downloads folder and run:

Bash

unzip VAPT_Tools_Linux.zip

4.3.3 Go inside the JADX Linux folder

1. Open **Terminal**.
2. After extraction click **three dots** and **open in terminal**



Or Navigate to the JADX **bin** directory inside the extracted Linux tools folder

Example:

Bash

```
cd ~/Downloads/VAPT_Android/VAPT_Tools_Linux/jadx/bin
```

If you extracted the folder somewhere else, update the path accordingly.

3. Confirm that the JADX GUI launcher file is present:

Bash

```
ls
```

Expected files include:

Text

```
Jadx
```

```
Jadx-gui
```

4.3.4 Give execution permission to JADX

1. Run the following command inside the `jadx/bin` folder:

Bash

```
chmod +x *
```

This allows Linux to run the JADX launcher scripts.

```
cdac@cdac-HP-Z2-Tower-G5-Workstation: /mnt/work/Downlo...
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-Tools
.Linux/jadx/bin$ chmod +x *
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-Tools
.Linux/jadx/bin$ ./jadx-gui
INFO - Checking for updates...
INFO - Update channel: STABLE, current version: 1.5.6
Gtk-Message: 09:02:36.603: Failed to load module "canberra-gtk-module"
INFO - No updates found
```

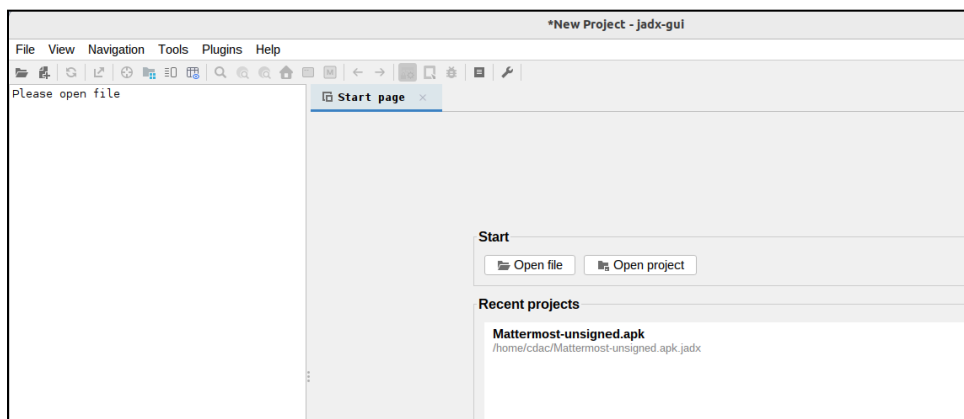
4.3.5 Open JADX GUI

1. Run the following command:

Bash

```
./jadx-gui
```

The JADX graphical interface should open.



4.4. Apktool

4.4.1 Verify Java installation

1. APKTool requires **Java**. Open a terminal and run:

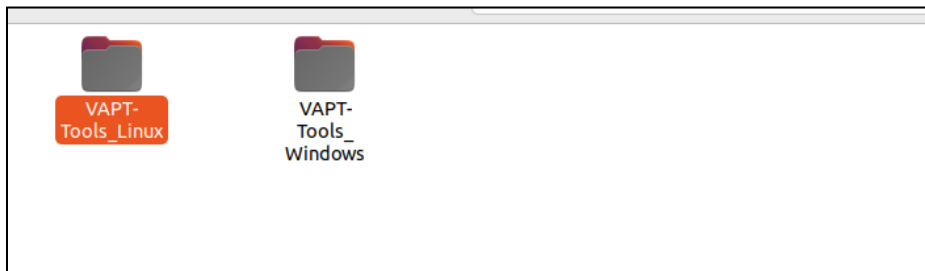
Bash

```
java -version
```

```
^Ccdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT_Linux/jadx/bin$ java -version
openjdk version "17.0.20" 2026-07-21
OpenJDK Runtime Environment (build 17.0.20+8-1-22.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.20+8-1-22.04-Ubuntu, mixed mode, sharing)
```

4.4.2 Open the extracted VAPT Linux tools folder

1. The **VAPT_Android_Tools.zip** file has already been downloaded and extracted.

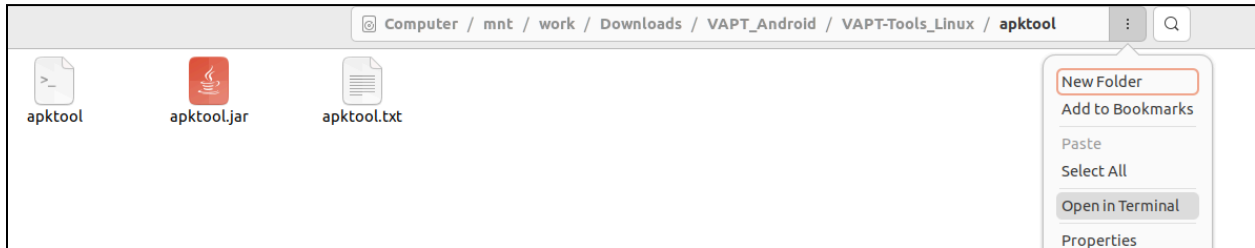


2. Open **Terminal** and go to the APKTool folder inside the extracted VAPT Linux tools directory.

If the ZIP was extracted in the **Downloads** folder:

Bash

```
cd ~/Downloads/VAPT_Android/VAPT_Linux_Tools/apktool
```



Note: Change the path according to the folder name and location on your Linux system.

4.4.3 Give execute permission to the APKTool launcher

1. Run the following command:

Bash

```
chmod +x *
```

4.4.5 Copy APKTool files to the system path

1. Copy the **apktool** launcher script and **apktool.jar** file to /usr/local/bin:

Bash

```
sudo cp -i apktool /usr/local/bin/
```

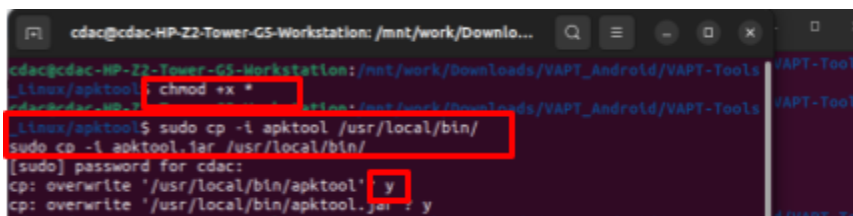
```
sudo cp -i apktool.jar /usr/local/bin/
```

2. Enter your **Ubuntu password** when prompted.
3. If Ubuntu asks whether to overwrite an existing file, type **Y**:

Text

Y

and press **Enter**, but only if you intend to replace the existing APKTool version.



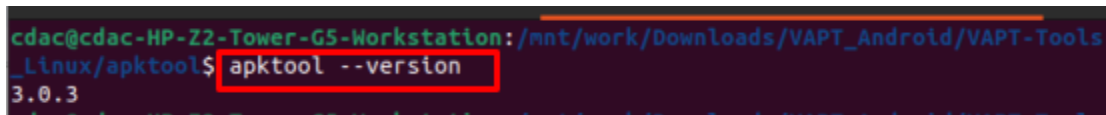
4.4.6 Verify the APKTool installation

1. Close the current terminal window.

Open a **new Terminal** window and run:

Bash

```
apktool --version
```



```
cdac@cdac-HP-Z2-Tower-G5-Workstation: /mnt/work/Downloads/VAPT_Android/VAPT-Tools
_Linux/apktools$ apktool --version
3.0.3
```

4.5. Adb Tool

4.5.1. The ADB tool is already included in the VAPT_Linux_tools file.

No separate download from the Android website is required.

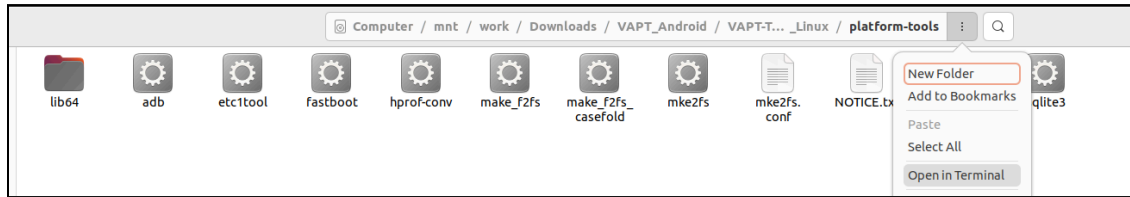
4.5.2. Open Terminal and go to the platform-tools folder inside the

extracted VAPT Linux tools folder.

Example:

Bash

```
cd ~/Downloads/VAPT_Linux/VAPT_Android_Tools/platform-tools
```



4.5.3. List the files in the folder:

Bash

```
ls -lh
```

The folder should contain the following file:

adb

4.5.4. Give execute permission to the ADB file:

Bash

```
chmod +x adb
```

4.5.5. Copy the ADB file to /usr/local/bin:

Bash

```
sudo cp -i adb /usr/local/bin/
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation: /mnt/work/Downlo...
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
_Linux/platform-tools$ ls -lh
total 20M
-rwxr-xr-x 1 cdac cdac 11M Jan 1 2008 adb
-rwxr-xr-x 1 cdac cdac 296K Jan 1 2008 etc1tool
-rwxr-xr-x 1 cdac cdac 3.2M Jan 1 2008 fastboot
-rwxr-xr-x 1 cdac cdac 14K Jan 1 2008 hprof-conv
drwxrwxr-x 2 cdac cdac 4.0K Sep 16 12:11 lib64
-rwxr-xr-x 1 cdac cdac 278K Jan 1 2008 make_f2fs
-rwxr-xr-x 1 cdac cdac 278K Jan 1 2008 make_f2fs_casefold
-rwxr-xr-x 1 cdac cdac 985K Jan 1 2008 mke2fs
-rw-r--r-- 1 cdac cdac 1.2K Jan 1 2008 mke2fs.conf
-rw-r--r-- 1 cdac cdac 1.2M Jan 1 2008 NOTICE.txt
-rw-r--r-- 1 cdac cdac 38 Jan 1 2008 source.properties
-rwxr-xr-x 1 cdac cdac 3.5M Jan 1 2008 sqlite3
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
_Linux/platform-tools$ chmod +x adb
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
_Linux/platform-tools$ sudo cp -i adb /usr/local/bin/
[sudo] password for cdac:
cp: overwrite '/usr/local/bin/adb'? y
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
```

4.5.6. Open a new Terminal window and verify the ADB installation:

Bash

adb --version

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
_Linux/platform-tools$ adb --version
Android Debug Bridge version 1.0.41
Version 36.0.0-13206524
Installed as /mnt/work/Android/Sdk/platform-tools/adb
Running on Linux 6.8.0-138-generic (x86_64)
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/VAPT_Android/VAPT-
_Linux/platform-tools$ adb devices
* daemon not running; starting now at tcp:5037
* daemon started successfully
List of devices attached
```

4.6. Python 3.12

4.6.1. Python is installed directly from the Ubuntu terminal. It is not included in the VAPT_Linux.zip file.

4.6.2. For Ubuntu 24.04, install Python 3.12 and pip using:

Bash

```
sudo apt update
```

```
sudo apt install -y python3.12 python3.12-venv python3-pip
```

4.6.3. For Ubuntu 22.04, first add the Python repository:

Bash

```
sudo apt update
```

```
sudo apt install -y software-properties-common
```

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

```
sudo apt update
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ sudo apt update
[sudo] password for cdac:
Hit:1 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ sudo apt install -y software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
Reading package lists... Done
Building dependency tree... Done
```

4.6.5. Install Python 3.12, virtual-environment support, development files, and pip:

Bash

```
sudo apt install -y python3.12 python3.12-venv python3.12-dev python3-pip
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ sudo apt install -y python3.12 python3.12-venv python3.12-dev python3-pip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

4.6.6. Verify the installation:

Bash

```
python3.12 --version
```

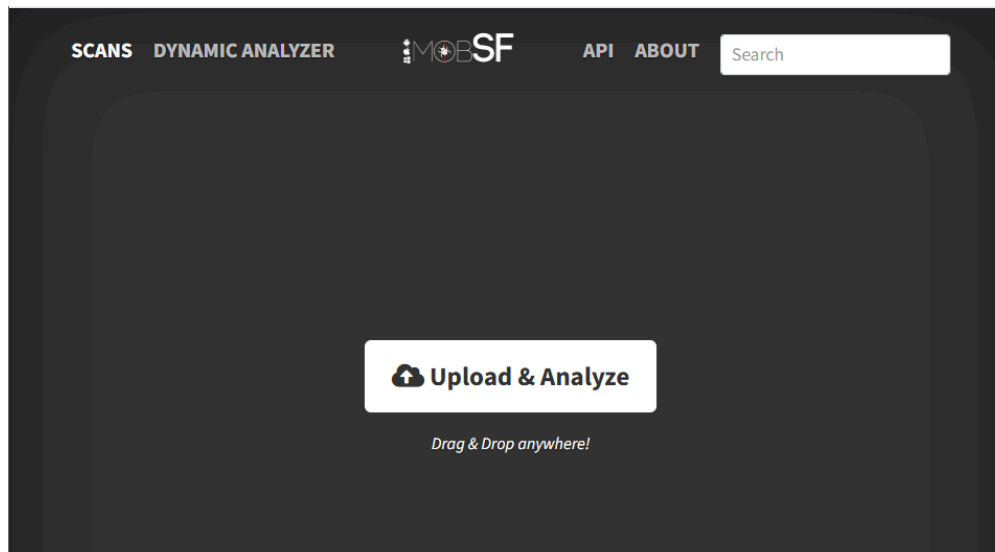
```
pip3 --version
```

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:~$ python3.12 --version
Python 3.12.13
```

4.7. MobSF

4.8.1. Go to <https://mobsf.live/>

4.8.2. Upload .apk file directly from the web application



5. Static Analysis for Windows, Linux

Note: All required static-analysis tools—JADX, APKTool, Android Studio, ADB, Python, and MobSF—have already been installed and verified. This section covers only the key analysis activities.

5.1 Select the Target APK

1. Download the apk in the **source code** folder Go to the folder containing the Android application. Example:
text

Downloads/AndroGoat_patched.apk

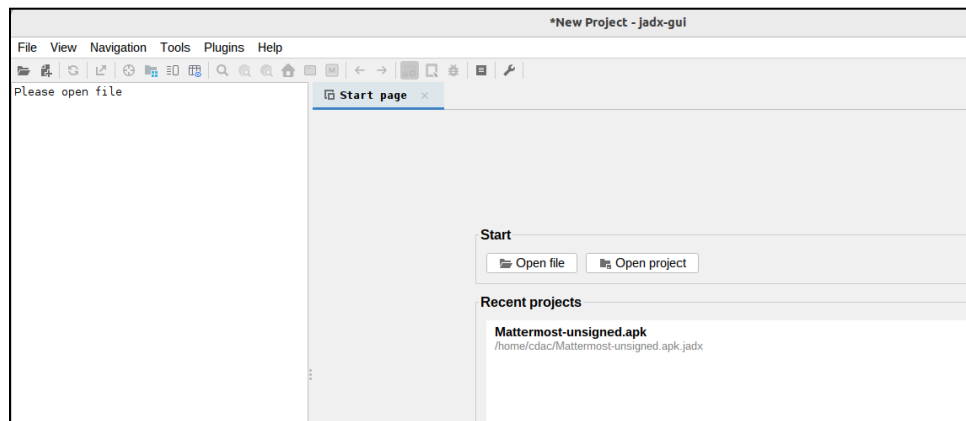
5.2 Open the APK in JADX

1. Launch **JADX GUI. Windows:**
 - Open the supplied **jadx-gui** folder.
 - Double-click **jadx-gui-<version>.exe**.



2. **Linux:**
 - Open Terminal inside the **JADX bin** folder location.
 - Run:
Bash
`./jadx-gui`

```
cdac@cdac-HP-Z3-Tower-G5-Workstation: /mnt/work/Downloads/VAPT_Android/VAPT-Tools
linux/jadx/bin $ ./jadx-gui
INFO - Checking for updates...
INFO - Update channel: STABLE, current version: 1.5.6
Gtk-Message: 09:02:36.683: Failed to load module "canberra-gtk-module"
INFO - No updates found
```

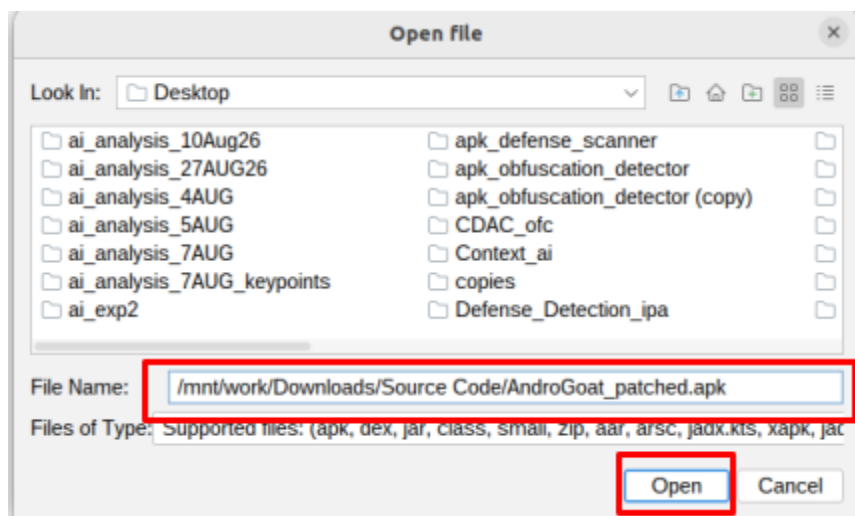


3. Click:
Text

Open file

4. Select the target APK, for example:
Text

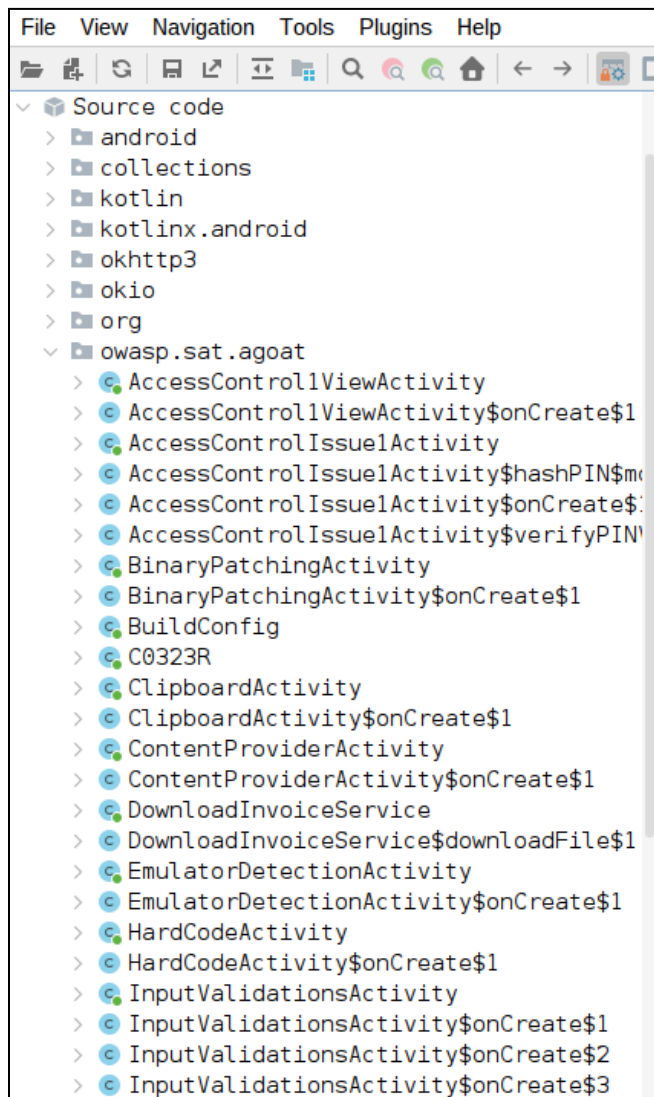
androgoat.apk

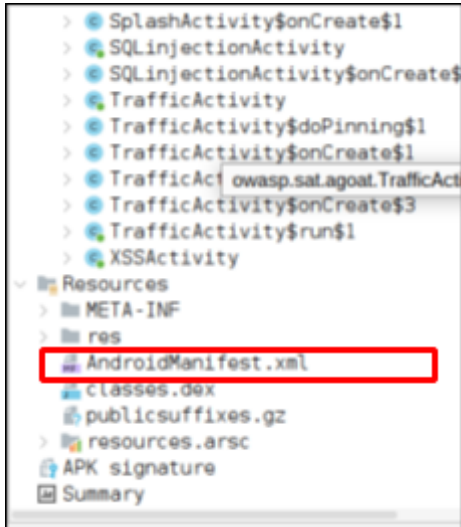


Note : keep the path according to apk location ,here it is /mnt/work/Downloads/Source Code/AndroGoat_patched.apk

5. Wait for **JADX** to decompile the application.

6. Expand the package tree on the left side and locate the application package.





7. Open important classes such as:

- Login or authentication activities
- WebView activities
- Network/API utility classes
- Database helper classes
- Shared Preferences utility classes

5.3 Search for Important Security Keywords

In JADX, press:

text

Ctrl + Shift + F

Search the entire project for the following keywords one at a time:

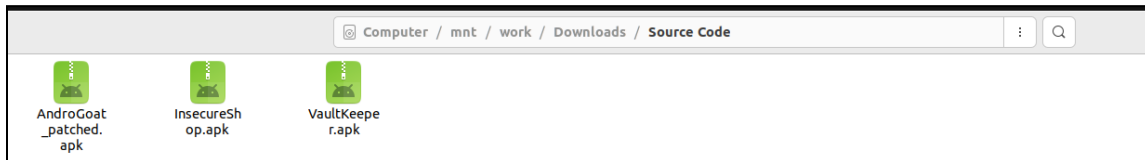
text

password
passwd
username
token
secret
api_key
apikey
client_secret
http://
https://
WebView
loadUrl
setJavaScriptEnabled
addJavascriptInterface
Log.
AES
DES
MD5
SHA-1
Random
SQLiteOpenHelper
SharedPreferences
root
Debuggable

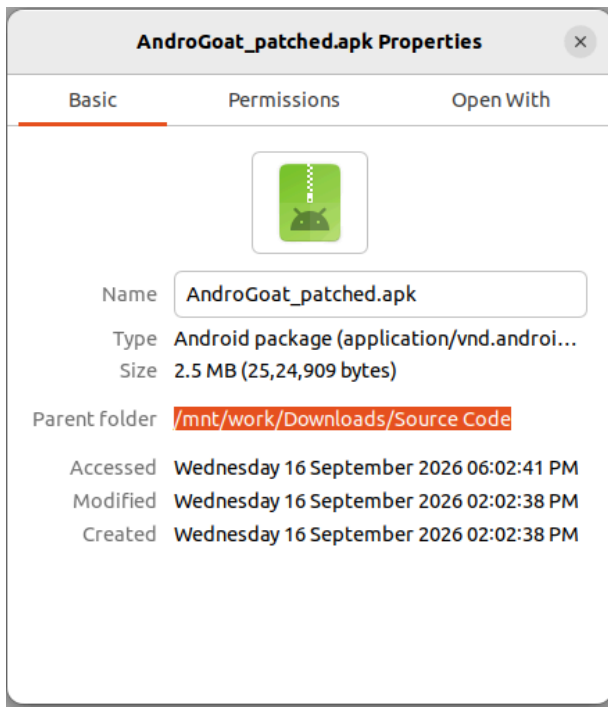


5.4 Decode the APK Using APKTool

1. Open PowerShell on Windows or Terminal on Linux in the folder that contains the **APK**.



For path (optional):



Run:

Bash

```
cd /mnt/work/Downloads/Source Code/
```

Here the the **apk path** is **:/mnt/work/Downloads/Source Code/AndroGoat_patched.apk**, keep the according to your apk location, then run below command

Bash

`apktool d shop.apk -o shop_decoded`

```
cdac@cdac-HP-Z2-Tower-G5-Workstation:/mnt/work/Downloads/Source Code$ apktool d
AndroGoat_patched.apk -o shop_decoded
I: Using Apktool 3.0.3 on AndroGoat_patched.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: /home/cdac/.local/share/apktool/framework/1
.apk
I: Decoding file resources...
I: Generating values XMLs...
```

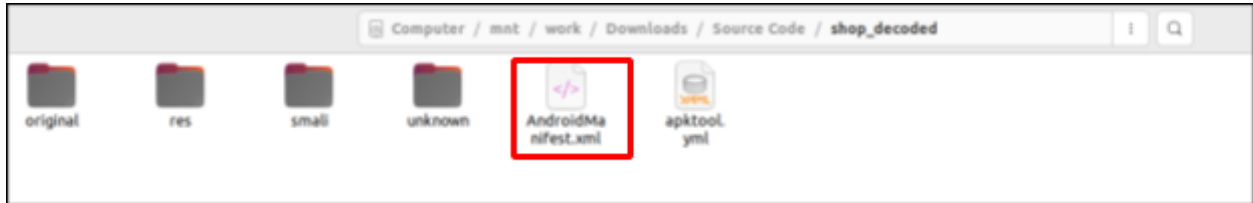
The decoded output folder should contain:

Text

AndroGoat/

- |— AndroidManifest.xml
- |— apktool.yml
- |— res/
- |— smali/
- |— assets/ (if present)
- |— lib/ (if present)

Replace **AndroGoat.apk** with the target APK file name.



5.5 Review AndroidManifest.xml

1. Open the decoded manifest file:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android" package="owasp.sat.agoat">
3   <uses-permission android:name="android.permission.INTERNET"/>
4   <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
5   <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_label"
6     android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/AppTheme">
7     <activity android:name="owasp.sat.agoat.SplashActivity">
8       <intent-filter>
9         <action android:name="android.intent.action.MAIN"/>
10        <category android:name="android.intent.category.LAUNCHER"/>

```

Text

shop_decoded/AndroidManifest.xml

2. Review the <application> tag and check the following properties:

Xml

Android:debuggable

android:allowBackup

android:usesCleartextTraffic

android:networkSecurityConfig

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android" package="owasp.sat.agoat">
3   <uses-permission android:name="android.permission.INTERNET"/>
4   <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
5   <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_label"
6     android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/AppTheme">
7     <activity android:name="owasp.sat.agoat.SplashActivity">
8       <intent-filter>
9         <action android:name="android.intent.action.MAIN"/>
10        <category android:name="android.intent.category.LAUNCHER"/>

```

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" package="owasp.sat.agoat">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/AppTheme">
        <activity android:name="owasp.sat.agoat.SplashActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity android:label="@string/app_name" android:name="owasp.sat.agoat.MainActivity">
        <activity android:label="@string/root" android:name="owasp.sat.agoat.RootDetectionActivity">
        <activity android:label="@string/logging" android:name="owasp.sat.agoat.InsecureLoggingActivity">
        <activity android:label="@string/xss" android:name="owasp.sat.agoat.XSSActivity">
        <activity android:label="@string/spi" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs">
        <activity android:label="@string/tempFile" android:name="owasp.sat.agoat.InsecureStorageTempActivity">
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue1Activity">
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue2Activity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

3. List all application components:

Xml

<activity>

<activity-alias>

<service>

<receiver>

<provider>

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android" package="owasp.sat.agoat">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/AppTheme">
        <activity android:name="owasp.sat.agoat.SplashActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity android:label="@string/app_name" android:name="owasp.sat.agoat.MainActivity">
        <activity android:label="@string/root" android:name="owasp.sat.agoat.RootDetectionActivity">
        <activity android:label="@string/logging" android:name="owasp.sat.agoat.InsecureLoggingActivity">
        <activity android:label="@string/xss" android:name="owasp.sat.agoat.XSSActivity">
        <activity android:label="@string/spi" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs">
        <activity android:label="@string/tempFile" android:name="owasp.sat.agoat.InsecureStorageTempActivity">
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue1Activity">
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue2Activity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
                <data android:host="vulnapp" android:scheme="androgoat"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

4. For each component, verify:
 - Whether it is exported.
 - Whether it has an intent filter.
 - Whether it is protected by a permission.
 - Whether the component exposes sensitive functionality.
5. Review all declared permissions:

Xml

```
<uses-permission>
```

6. Identify unnecessary, excessive, dangerous, or security-sensitive permissions.

5.6 Review Deep Links and WebViews

1. In **AndroidManifest.xml**, search for:

Xml

```
<intent-filter>
```

```
<data
```

```
Android:scheme
```

```
Android:host
```

```
Android:path
```

2. Record every custom **URI scheme** and deep-link handler.

Example:

Xml

```
<data android:scheme="inseureshop"
```

```
android:host="com.inseureshop" />
```

3. In JADX, trace the component that receives the URI.

4. For WebView-related classes, verify whether the application uses:

Java

```
webView.loadUrl(...)
```

```
webView.getSettings().setJavaScriptEnabled(true)
```

```
webView.addJavascriptInterface(...)
```

5.7 Review Resources and Sensitive Files

1. Review the decoded files and folders:

Text

```
shop_decoded/res/values/strings.xml
```

```
shop_decoded/res/xml/
```

```
shop_decoded/res/raw/
```

```
shop_decoded/assets/
```

Search for:

text

password

token

secret

api_key

server URL

database name

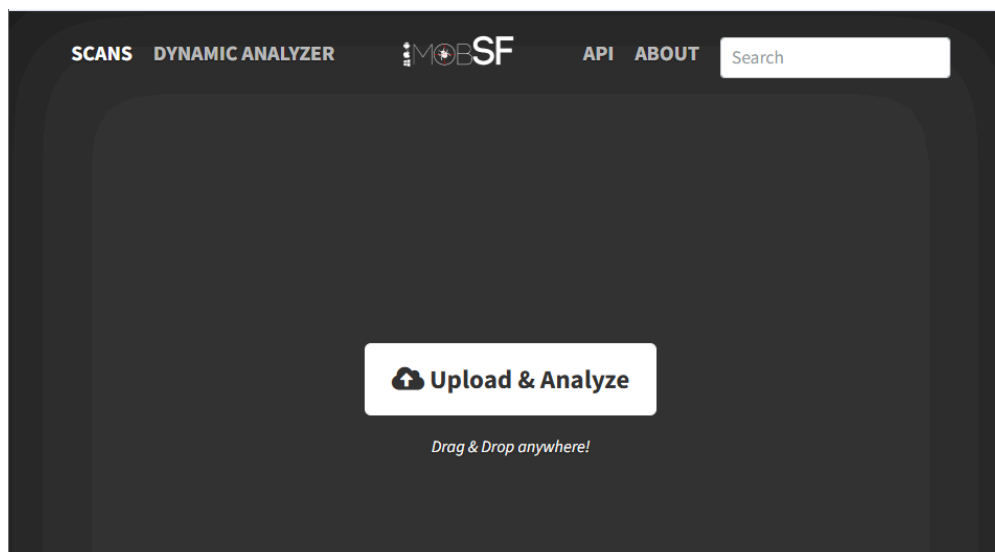
encryption key

certificate
private key
HTTP endpoint
debug configuration

5.8 Scan the APK Using MobSF

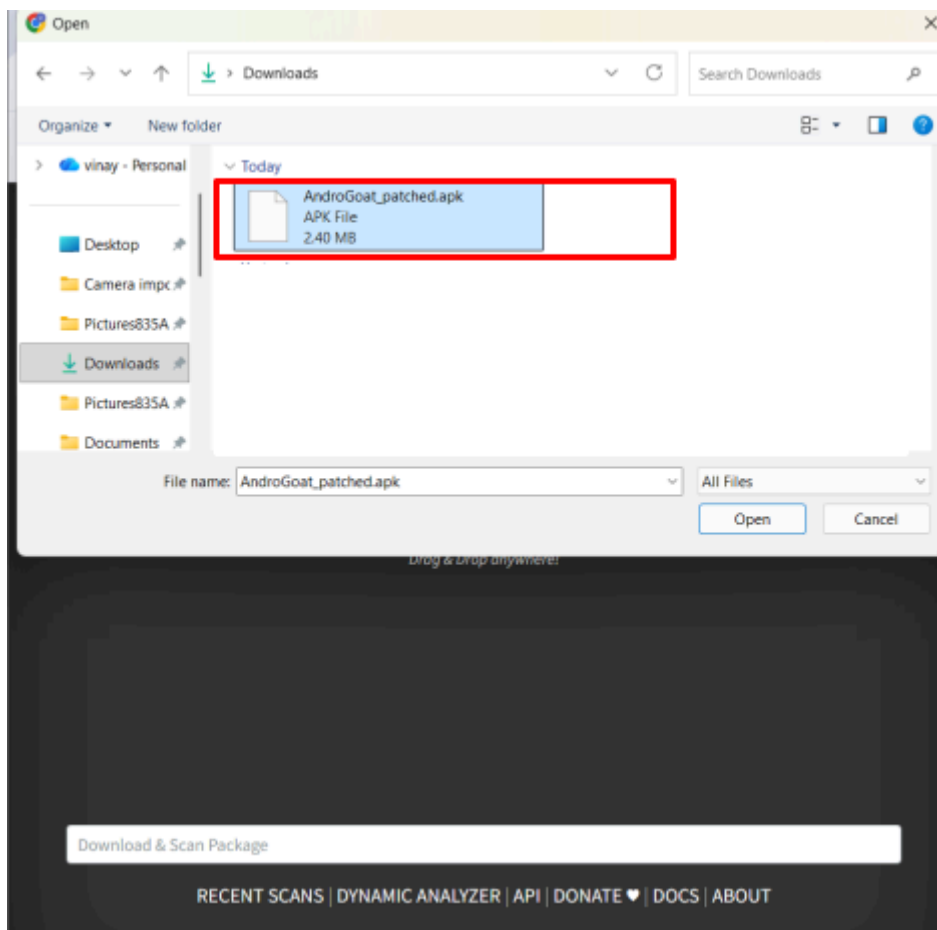
1. Open the approved MobSF instance. For an instructor-approved training APK, you may use:
Text

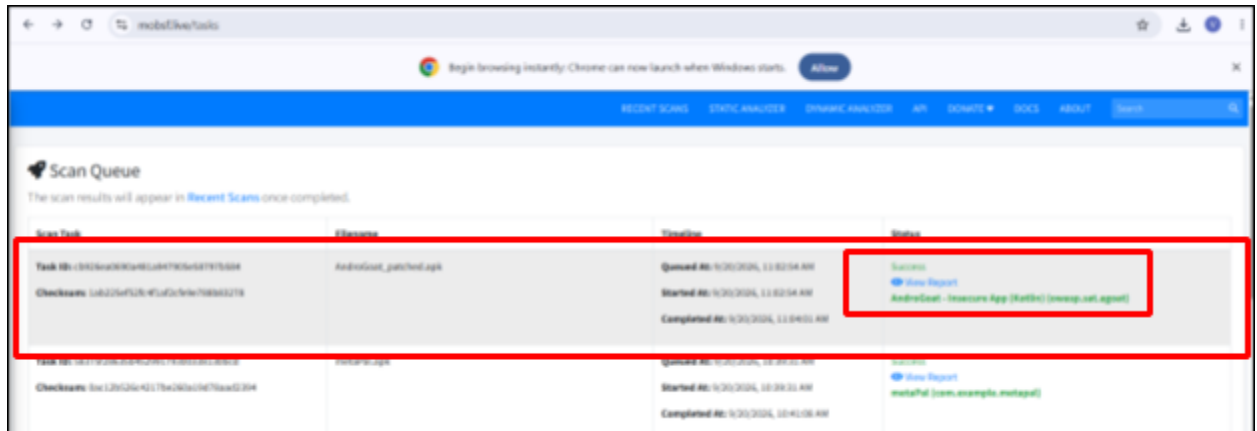
<https://mobsf.live/>



2. Select:
Text
Upload & Analyze

3. Upload the target **APK** ,it takes time once the analysis completed **click** on the **view report**





Scan Task	File Name	Metadata	Status
Task ID: c1d3eac0000000000000000000000000 Checksums: 1a6225ef52b4f5a2a2e5e7e5e5e5e5e5	Android.apk	Queued At: 11/20/2024, 11:02:04 AM Started At: 11/20/2024, 11:02:04 AM Completed At: 11/20/2024, 11:04:01 AM	Success View Report Android - Insecure App (Kotlin) (mewap.net.apk)
Task ID: 00000000000000000000000000000000 Checksums: 00000000000000000000000000000000	Android.apk	Queued At: 11/20/2024, 11:04:01 AM Started At: 11/20/2024, 11:04:01 AM Completed At: 11/20/2024, 11:04:01 AM	Success View Report metaPul (com.example.metapul)

4. Review the following sections:
 - Application Information
 - Security Score
 - Manifest Analysis
 - Code Analysis
 - Permissions
 - Network Security
 - Certificate Analysis
 - Trackers and Third-Party Libraries
5. Compare MobSF results with your manual findings from JADX and APKTool.
6. Do not report a MobSF alert without reviewing the referenced code, manifest, or resource file.

5.9 Identify Security Findings and Prepare Observations

Steps:

1. Select another APK provided or an APK from an external source for analysis.
2. Perform the static analysis using the tools and techniques covered in this lab, including **JADX, APKTool, AndroidManifest.xml analysis, Deep Link and WebView analysis, resource and sensitive file analysis, and MobSF.**
3. Identify and document any **security-relevant findings** observed during the analysis.
4. For each finding, prepare an observation containing:
 - **Finding/Issue**
 - **Location** (class, method, file, or manifest entry)
 - **Evidence** such as code snippet, configuration, or screenshot
 - **Security impact**
 - **Observation/Conclusion**
5. Prepare a brief **security analysis report** summarizing the findings identified in the selected APK.